TEMCO LEARN

Test1

1.0 · 5 pages · built Nov 2, 2026

1. Test1 documentation
2. Getting started
3. Ports and wiring
4. Writing a program
5. Alarms and how to clear them

Test1 documentation

What Test1 is, what it answers, and how this documentation is arranged.

Test1 is the controller every later Temco product was measured against: a small, fixed set of inputs and outputs, one program, and a display that says what the machine is doing. This documentation covers the version you are holding, and nothing else — each version keeps its own pages, so a machine commissioned two years ago still reads correctly.

What this version covers

- Wiring an input and proving it changed
- Pointing a port at the right device
- Writing and downloading a program
- Reading the alarm log, and clearing an alarm that has gone

What it does not cover

Networking and protocol work belong to BACnet and the tools that speak it. If you are looking for how a supervisor reads these points, start there rather than here.

> Every page in this version was written against firmware 1.0. A page that only applies to a later build says so at the top.

Getting started

Install the driver and prove the controller answers before anything else.

Work through this page at the machine, not at a desk. It takes about ten minutes and ends with a controller that answers, which is the state every other page assumes.

> **NOTE**
>
> Work at the controller, not at a desk: the driver install needs the machine plugged in, and the last step asks the hardware a question rather than the software.

Install the driver

The driver ships inside the Temco installer. Run it as an administrator, then plug the controller in. Windows shows a new port under **Ports (COM & LPT)** — the number beside it is the one you will point at in the next step.

Point the port

Open the tool, choose the port the controller appeared on, and press **Connect**. A controller that has never been commissioned answers on its own address; one that has been on a network answers on the address it was given.

If nothing answers, try the other ports in the list before assuming the controller is faulty: a USB adapter usually takes the port beside its own.

Check the answer

Ask for the controller's identity. A good answer names the model, the firmware and how long the machine has been running:

```text
Test1  firmware 1.0  uptime 0d 00:07:12
```

That line is enough to prove three things: the loop is powered, the driver is talking to it, and the firmware in front of you is the firmware this documentation was written for.

Where next

- Ports and wiring — what each terminal is for
- Writing a program — the first thing a machine needs to do
- Alarms — when the machine says something is wrong

Ports and wiring

What each terminal is for, and the two mistakes that account for most commissioning calls.

The controller has the same terminals on every model in the family. The numbers are printed beside them; the words below are what they do.

The terminals

Terminal	Name	What it is for
1–8	Universal inputs	A switch, a sensor or a 0–10 V signal; the tool decides which
9–14	Digital outputs	Relays, 24 V AC or DC, one amp each
15–16	Analog outputs	0–10 V, for a damper or a valve
17–18	Power	24 V AC, either polarity; the controller does not care

Two mistakes worth knowing

A universal input set to the wrong kind reads plausibly. A 0–10 V sensor wired into an input the tool believes is a switch will read "closed" for ever, and nothing complains. Set the kind before wiring, and confirm it in the tool.

A shared 24 V supply is not shared ground. Two controllers on one transformer share a path that will carry current the wrong way when one of them is switched off. Either give each controller its own transformer, or bond the grounds deliberately — never by accident.

Prove an input

After wiring, change the input by hand and watch the value in the tool. A reading that moves is the only proof that matters; a reading that sits still is worth investigating before the panel goes back on.

Writing a program

The shape of a Test1 program, and the one rule that keeps it readable a year later.

A program is a list of lines the controller runs top to bottom, ten times a second. That is the whole model: no tasks, no threads, and no reason to fear it.

The shape of a program

```text
10  IF IN1 = ON THEN OUT1 = ON
20  IF IN1 = OFF THEN OUT1 = OFF
30  IF AI1 > 22.5 THEN OUT2 = ON
```

Every program worth keeping has three parts: what it reads, what it decides, and what it writes. Put them in that order and a stranger can read it. Mix them and only the person who wrote it can.

The rule

One output is set in one place. Two lines that both write `OUT1` will, sooner or later, disagree — and the machine will do something nobody intended at a moment nobody predicted. If two conditions can drive one output, write one line that decides between them.

Download and watch

Downloading replaces the running program. The controller keeps running its old program until the download completes, then switches — so a download is not a gap in control, and a failed one leaves the old program in place.

Watch the first ten seconds after a download. A program that was going to misbehave almost always shows it in the first scan of the inputs.

Alarms and how to clear them

What an alarm means, where the log is, and when clearing one is the wrong thing to do.

An alarm is a sentence the controller writes when a value crosses a limit it was given. It is not a diagnosis, and it is not a fault: it is a fact with a time beside it.

The log

The controller keeps the last 100 alarms in order. Each entry holds the time, the alarm, and whether it has since cleared. The log survives a power cycle, which is what makes it useful on a Monday morning.

Clearing an alarm

An alarm clears itself when the value returns inside its limit. Pressing **Clear** does not fix anything — it hides the entry from the active list and leaves it in the log, so the next person sees a machine that is quiet rather than one that was fixed.

Clear an alarm when you have dealt with the cause and want the machine to look like it. Investigate first; the log entry is the only record you will have.

The three that come up most

Alarm	What it usually is
Sensor open	A wire off, or a sensor that has failed open — check at the terminal, not in the tool
Over temperature	A fan that has stopped, or a filter nobody has changed
Program stopped	The controller restarted; look at the log for what happened before it did